

EvoODE — Projektjournal

Stand: 2026-07-30 · Zeitraum: 2026-04-20 bis 2026-07-30

0. Wie dieses Dokument zu lesen ist

Dieses Journal ist der **rote Faden**: Es erzählt, was gebaut wurde, was verworfen wurde, warum, und was die Zahlen dazu sagen. Es ersetzt keines der bestehenden Dokumente, sondern verbindet sie:

Dokument	Rolle
CLAUDE.md	Zustandsbeschreibung: Architektur, Konventionen, aktuelle Prioritäten
DIARY.md	Chronologisches Protokoll, neueste Einträge zuerst, mit Commit-Hashes
PAPER_1.md	Ausführungsplan für Paper 1, maßgeblich bei Widersprüchen
dieses Journal	Narrativ und Begründungen: der Zusammenhang zwischen den Einträgen

Alle Zahlen stammen aus protokollierten Messungen. Wo eine Aussage eine Vermutung ist, steht das ausdrücklich dabei.

1. Das Projekt in einem Absatz

EvoODE ist ein Julia-Forschungsframework zur datengetriebenen Entdeckung interpretierbarer ODE-Systeme. Der Ansatz unterscheidet sich von SINDy (feste Bibliothek, direkte Regression) und von klassischer genetischer Programmierung (globale Suche aus großen Zufallsstrukturen): EvoODE **beginnt minimal und wächst inkrementell**, und erhöht Komplexität nur, wenn einfachere Strukturen nicht ausreichen. Der wissenschaftliche Kern ist nicht das Fitten von ODEs, sondern die Frage, wie Modellstrukturen aufgebaut, erweitert, validiert und kontrolliert werden sollten.

Forschungsfokus der Dissertation: **effiziente und robuste Suchstrategien für die interpretierbare Entdeckung gekoppelter ODE-Systeme.**

2. Der Zeitstrahl in Phasen

Zeitraum	Phase	Ergebnis
2026-04-20 – 04-22	Fundament	Kernpipeline, EvoGrow v1/v2.1/v2.2, GP-Baseline, Experimentinfrastruktur
2026-04-22 – 05-08	Phase-A-Experiment	300 Läufe, 4,5 Tage Rechenzeit, 300/300 erfolgreich
2026-05-08 – 05-11	Auswertung & Freeze	H1 partial, H2 supported, H3 partial, H4 vakuös
2026-05-17	Strategischer Pivot	Paper 1 wird kein Pretuning-Paper; Gate-Struktur eingeführt
2026-05-30	Gate 1	v2.2 nicht paper-ready → EvoGrow v3 ausgelöst
2026-07-20	v3-Entwurf + Regressionshistorie	Designnotiz, Lockstep-Brücke, append-only Historie
2026-07-22	Kostenzusammenbruch	Volllauf abgebrochen; 62 % der Rechenzeit oberhalb der nötigen Stage
2026-07-22 – 07-23	Screening & Numerik	Screening verworfen, wieder aufgenommen; Numerikbefund; Faktor 6,2
2026-07-29	WP-T2 — die Entscheidungsmessung	Overshoot auf System 26 toleranzinvariant → algorithmisch, v3 bestätigt; Screening ohne Discovery-Gewinn
2026-07-29 – 07-30	v3-Kette scharf geschaltet	v3.3 Kindergenerierung, v3.4 Promotion, v3.5 Metriken; divergenter Pfad erstmals gelaufen
2026-07-30	Zweiter Kostenzusammenbruch	Baseline-v1-Matrix ~2 Wochen, 2/3 Wegwerf-Compute; abgebrochen; Scope-Entscheidung offen

3. Die Chronik

3.1 Fundament (2026-04-20 bis 2026-04-22)

Die Kernpipeline `discover()` steht: Daten → Struktur → Parameter → Simulation → Bewertung → Iteration. Implementiert wurden EvoGrow (v1 flach, v2.1 gestuft, v2.2 stage-lokal), eine GP-Baseline, `StagedPolynomialBasis` mit fünf Komplexitätsstufen (linear, selbst-quadratisch, paarweise Kreuzterme, selbst-kubisch, trigonometrisch), `MSELoss` und `BFGSOptimizer`.

Zwei Designachsen wurden bewusst getrennt gehalten und dürfen nicht wieder zusammengelegt werden:

- **Stage Progression Policy** — wann wird eine Stage gehalten, promoviert oder beendet
- **Stage Usage Policy** — wie stark werden neu freigeschaltete Terme danach bevorzugt (`:hard` , `:soft` , `:passive`)

Ebenfalls hier entstanden: `pretune.jl` (Least-Squares-Warmstart über finite Differenzen) und die Experimentinfrastruktur mit Manifest-Generator, sequentiellem Runner und Aggregator, inklusive Per-Run-

Dateiprotokoll mit atomaren Writes.

3.2 Das Phase-A-Experiment (2026-04-22 bis 2026-05-08)

`paper1_phaseA_v1` : 10 Systeme × 6 Varianten × 5 Seeds = **300 Läufe**, Laufzeit **4,5 Tage**, Ergebnis 300/300 erfolgreich.

Nebenbefund vom 2026-04-27, der später noch wichtig wird: `profile_init.jl` hing **über 48 Stunden** in einem einzelnen BFGS-Aufruf (Lorenz 3D). Reaktion war die Einführung von `time_limit_s = 300.0` als Wall-Clock-Notbremse im `BFGSOptimizer` — begründet mit „~100× der medianen Aufrufzeit, greift bei normalen Läufen nie“. Diese Annahme erwies sich knapp drei Monate später als falsch (siehe 3.8).

3.3 Auswertung und Freeze (2026-05-08 bis 2026-05-11)

Hypothesenverdikte:

Hypothese	Verdikt	Belegt auf
H1 Stage-Overshoot-Reduktion	PARTIAL	nur System 54 (1 von 6)
H2 kompetitive Rückgewinnungsqualität	SUPPORTED	7 von 8 Systemen
H3 Reduktion verschwendeter Level	PARTIAL	Systeme 11 und 54 (2 von 6)
H4 Ordnung der Usage-Policies	vakuös	alle <code>exact_match = 0</code> , Ordnung nicht unterscheidbar

Das stärkste Ergebnis war ein anderes — EvoGrow gegen die GP-Baseline auf gekoppelten Systemen:

System	Beste EvoGrow	GP	Faktor
31 SIR	7,0e-05	0,314	~4.500×
54 Lorenz	7,4e-04	0,921	~1.200×
26 Lotka-Volterra	2,5e-04	2,98e-03	~12×

Gleichzeitig eine bekannte Schwäche: System 11 ($du = -u^3$) erreicht Loss $\sim 4,4e-15$, aber `exact_match = 0`, weil Wachstum ohne Pruning Nullterme akkumuliert. Als echter algorithmischer Mangel dokumentiert, nicht als Metrikfehler.

Die Phase-A-Ergebnisse sind **eingefroren** und werden für die finalen Paper-Aussagen nicht mehr verwendet.

3.4 Strategischer Pivot (2026-05-17)

Paper 1 wird **kein** Pretuning-Vergleichspaper. Pretuning wird für Paper 1 vollständig deaktiviert (`use_pretuning = false`) und als mögliches Folgepaper zurückgestellt.

Neue Kernfrage: *Ist inkrementelles, gestuftes Wachstum ein effektiver Suchmechanismus für interpretierbare ODE-Entdeckung — und wo hilft es, wo versagt es?*

H1/H3 „PARTIAL“ wird umgedeutet: nicht als schwaches Ergebnis, sondern als **diagnostisches Signal**, dass systemweites Staging für mehrdimensionale Systeme zu grob sein könnte. Es wird eine Gate-Struktur

eingeführt (Gate 1: Ist v2.2 paper-ready? Gate 2: Ist v3 paper-ready?).

3.5 Gate 1 — v2.2 scheitert (2026-05-30)

15 Diagnostikläufe, v2.2 stage_local, `use_pretuning = false`, `n_levels = 30`, 3 Seeds je System.

System	Ergebnis	Loss	Stage	Laufzeit
3 Logistic	<code>pruned_match = true</code> , alle Seeds	$\sim 7e-10$	Overshoot 0–3	—
11 Cubic	<code>pruned_match = true</code> , alle Seeds	$\sim 4e-15$	4/4	$\sim 1,3$ s
26 Lotka-Volterra	false , alle Seeds	$5e-4 - 1,4e-3$	5/3	2.800 – 9.400 s
31 SIR	false , alle Seeds	$7e-11 - 1e-4$	5/3	—
63 SEIR	false , alle Seeds	$9e-4 - 1,8e-3$	5/3	11.000 – 31.000 s

Auf System 26 lautet die gefundene Struktur $du_1 = 5,05 \cdot u_1 - 3,87 \cdot u_1 \cdot u_2$, $du_2 = 1,13 \cdot u_1 - 1,80 \cdot u_1^2$ — die Terme sind komplett falsch. Der Loss reißt in Stage 3 auf $\sim 1e-3$ auf, und danach verbessert die Eskalation bis Stage 5 nichts mehr; trigonometrische Terme sind hier nutzlos.

Diagnose: Der systemweite Staging-Mechanismus zwingt alle Gleichungen gemeinsam in höhere Stages, sobald der globale Fortschritt stagniert. Das ist kein Metrikfehler, sondern eine echte algorithmische Schwäche.

Entscheidung: v2.2 ist nicht paper-ready. Phase 2 (EvoGrow v3, gleichungsweises Staging) wird ausgelöst.

Nebenbeobachtung, noch offen: Auf System 31 Seed 42 wird bei Loss $\sim 7e-11$ ein Spurious-Term $0,0022 \cdot u_1$ nicht weggeprunt, weil die Schwelle $1e-3 \times \text{max_coeff} = 4e-4$ darunter liegt.

3.6 v3-Entwurf und Lockstep-Brücke (2026-07-20)

WP-v3.1 liefert `docs/evogrow_v3_design.md`: gleichungsweise Stage-Zustände (`eq_stages`, `eq_levels_in_stage`, `eq_plateau_histories`), ein Fortschrittssignal auf Basis des Ableitungsresiduums pro Gleichung, eine gleichungsweise Promotionsregel, gleichungsweise Kindergenerierung und neue Metriken.

WP-v3.2 implementiert `EvoGrowV3` bewusst als **Lockstep-Brücke**: Die Variante führt bereits gleichungsweise Zustände, promoviert aber alle Gleichungen gemeinsam und ist dadurch verhaltensgleich zu v2.2. Verifiziert als bit-identisch auf System 3 und 26. Das ist der Regressionsanker, gegen den alle späteren v3-Schritte geprüft werden.

3.7 Regressionshistorie (2026-07-20)

Auf Wunsch entstand eine **longitudinale Historie**, um zu verfolgen, ob Ergebnisse besser oder schlechter werden:

- `studies/regression/history.jsonl` — append-only, versioniert, ein Record je Zelle
- Schlüssel: (`git_hash`, `variant`, `system`, `seed`) plus `config_fingerprint` (SHA256 über die metrikrelevante Konfiguration, erste 16 Hex-Stellen)
- Suite: 5 Systeme (3, 11, 26, 31, 63) $\times$ 3 Seeds (42, 123, 7) $\times$ 2 Varianten = 30 Zellen

Dazu kam Fortschrittslogging: gestapelte Fortschrittsbalken (`ProgressMeter`), äußerer Balken über alle Zellen, innerer Balken über die Level eines Laufs, Detailausgabe nur in `run.log` . Leitlinie war ausdrücklich „so wenig wie möglich, aber so viel wie nötig“.

Ein Rechnerneustart beendete den ersten Volllauf nach 7 von 30 Records. Weil die Historie append-only ist, überlebten die 7. Daraufhin **WP-H1d (Resume)**: Der Runner überspringt beim Neustart alle erfolgreichen Zellen desselben `config_fingerprint` . Bewusst **nicht** auf `git_hash` geschlüsselt, damit ältere Records bei unveränderter Konfiguration weiterhin erkannt werden.

3.8 Der Kostenzusammenbruch (2026-07-22)

Der zweite Volllauf wurde nach **23 von 30 Zellen und 40,5 Compute-Stunden abgebrochen** — die Laufzeit war nicht mehr vertretbar.

Befund 1 — der Anker hält, mit einer Ausnahme. v2.2 und v3.2 waren in 7 von 8 überlappenden Zellen bit-identisch. Die Abweichung (System 26 Seed 123: 1,3916e-3 gegen 1,3713e-3) war **kein v3-Bug**, sondern ein Reproduzierbarkeitsleck: `BFGSOptimizer` wurde ohne `time_limit_s` konstruiert, der Default von 300 s Wall-Clock wurde an `Optim.jl` durchgereicht. Damit hing die Zahl der Optimierer-Iterationen von der Maschinenlast ab. Dieselbe Zelle brauchte 13.352 s gegen 20.158 s.

Befund 2 — das Kostenprofil. Rekonstruiert aus `run.log` über alle 23 Zellen:

Stage	Level	Zeit	Anteil	s/Level
1	124	0,7 h	1,9 %	21
2	129	6,1 h	16,3 %	170
3	94	11,6 h	30,9 %	443
4	55	7,4 h	19,7 %	482
5	48	11,7 h	31,3 %	878

Ein Stage-5-Level kostet das **42-fache** eines Stage-1-Levels. Pro Zelle oberhalb der erwarteten Stage aufsummiert:

24,9 von 40,5 Stunden — 62 % der gesamten Rechenzeit — wurden in Komplexität investiert, die die Systeme nie gebraucht haben.

Auf gekoppelten Systemen liegt der Anteil bei 38–80 %. Extremfall System 63 Seed 123: 8,4 h Laufzeit, davon 6,7 h verschwendet; ein einzelnes Level (16, Stage 4) kostete 3,4 Stunden.

Konsequenz: Laufzeitproblem und Gate-1-Failure-Mode sind **dasselbe Problem**. Die Suche läuft über die nötige Stage hinaus, und jede zusätzliche Stage ist überproportional teurer, weil kubische und trigonometrische Terme steife und divergierende Kandidaten-ODEs erzeugen.

Blocker für Phase B: 63 Systeme × 2 Bedingungen × 3 Seeds = 378 Läufe, davon 240 gekoppelt. Bei ~3,5 h je gekoppelter Zelle sind das **über 800 Stunden ≈ 5 Wochen durchgehend**. Phase B war mit diesem Kostenprofil nicht durchführbar.

3.9 WP-P1 — Determinismus, Budgets, Instrumentierung

Drei Runden (P1, P1b, P1c), jeweils mit Review:

- **Determinismus:** Wall-Clock aus dem Ergebnispfad entfernt (`time_limit_s = 86.400 s` als reine Notbremse, deren Auslösen gezählt wird).
- **Getrennte Budgets** für Screening während der Suche und finale Validierung, mit den heutigen Werten als Default, damit ohne Aktivierung kein Verhalten sich ändert.
- **Instrumentierung** pro Level: Fits, Integrationen, verworfene Integrationen, Zeitanteil Optimierung gegen Simulation, Retcode-Kategorien.

Zwei Reviewbefunde waren substantiell:

1. `isoutofdomain` **war das falsche Primitiv.** Belegt in den DiffEq-Quellen (`OrdinaryDiffEqCore/.../integrator_utils.jl:268-286`): Es verwirft den Schritt und verkleinert `dt` , statt abzuberechnen — für divergierende Kandidaten also **teurer**. Das Abbruch-Primitiv ist `unstable_check` , dessen Default bereits `any(!isfinite, u)` prüft. Umgestellt.
2. **Der erste Benchmark-Zuschnitt hätte am Problem vorbeigemessen.** Mit 12 Leveln kostet System 26 nur 1,6 min; der Kostenausbruch beginnt bei Level 13 (147 s), 14 (878 s), 16 (611 s), 19 (1.484 s). Budget auf 18 Level korrigiert.

Ergebnis des Mikro-Benchmarks (System 26, Seed 42, 18 Level, je 370 Parameter-Fits):

	A Referenz	B Screening-Budgets	Faktor
Laufzeit	3.222,6 s	1.189,8 s	2,71×
Integrationen	1.741.484	2.488.973	0,70×
Kosten pro Integration	1,763 ms	0,409 ms	4,31×
Solve-Anteil an Laufzeit	95 %	86 %	
Overhead ohne Solve	153 s	166 s	

Sauberster Einzelvergleich: Stage 2, in beiden Fällen exakt 8 Level — A 2.937,9 s, B 297,1 s, **Faktor 9,9**.

Die Zahl, die alles Weitere bestimmt hat: 95 % der Laufzeit stecken in der ODE-Integration, der Overhead außerhalb beträgt 153 s. Ein Screening ohne Integration hätte damit eine Obergrenze von **~21×** (3.222,6 / 153).

Zwei Nebenfunde mit eigener Bedeutung:

- A ist bit-identisch zu Baseline v0 (`0.001391623174905009`) — aber v0 brauchte dafür 30 Level und 3,0 h und lief bis Stage 5, A erreicht denselben Loss in 18 Leveln und 53,7 min bei Stage 3. **Die Level 19–30 kosteten rund 2,1 h und verbesserten nichts.**
- Über alle 23 v0-Zellen: 13 liefen über Level 18 hinaus, und in **allen 13** war der Loss bei Level 18 bereits identisch zum Endergebnis. Das entspricht **15,8 von 40,5 h = 39 %** der Rechenzeit ohne jede Verbesserung.

Eine Kürzung des Level-Budgets wurde trotzdem abgelehnt. Der Loss bliebe identisch, `final_stage` , `stage_overshoot` und `wasted_levels` aber nicht: System 26 Seed 42 zeigt bei 30 Leveln Stage 5 / Overshoot 2, bei 18 Leveln Stage 3 / Overshoot 0. Ein Level-Cap würde das Overshoot-Phänomen **wegschneiden statt messen** — also genau das, was v3 beheben soll.

3.10 Die Screening-Spur (2026-07-22 bis 2026-07-23)

Idee: Die 95 % Solve-Anteil lassen sich nur angreifen, wenn die *Anzahl* der Integrationen sinkt. Kandidaten werden deshalb über ein **Ableitungsresiduum in geschlossener Form** bewertet (die Maschinerie liegt bereits in `pretune.jl`), und nur eine kleine Auswahl wird simuliert.

WP-P2.1 (Designnotiz) legte fest, dass Plateau-Erkennung, Stoppllogik und Stage-Promotion weiterhin ausschließlich auf dem **simulierten** Loss arbeiten und der berichtete `loss` ein voll simulierter Loss bleibt. Das Review ergänzte das fehlende Kostenmodell:

Pro Level (20,6 Kandidaten)	Kosten	Faktor
heute: alle per BFGS mit Simulation	170,9 s	1,0×
Top 10 per BFGS	83,0 s	2,1×
Top 10, je 10 Polish-Iterationen	4,2 s	41×
Top 10, je 20 Polish-Iterationen	8,3 s	21×

Daraus die Architektur: **begrenztes Nachpolieren** statt roher LS-Parameter, weil `plateau_tol = 1e-4` und `loss_tol = 1e-8` auf BFGS-optimierte Losses kalibriert sind.

Der Verlauf war holprig und lehrreich:

1. **WP-P2.2/2b:** Drei Blocker im Review — eine Feldkollision, die Records unbrauchbar gemacht hätte; ein falscher Zähler für die Budget-Erschöpfung; und eine Rangübereinstimmung, die ihre eigene Frage nicht beantworten konnte, weil sie nur über die *ausgewählten* Kandidaten rechnete.
2. **WP-P2.2c (erste echte Zahlen, Toleranz 1e-6):** System 11 funktioniert (rho **+1,0**, 1,53× schneller, korrekte Struktur). System 3 versagt: rho **-0,78**, Loss 3,24e-8 statt 2,66e-10, Stage 5 statt 3, und mit 0,89× sogar **langsamer** als der Referenzpfad.
3. **WP-P2.3 (AIC):** Ergebnis bit-identisch zur Vorrunde, rho unverändert -7/9. Grund, im Nachhinein rechnerisch belegt: Bei $n = 200$ beträgt die AIC-Strafe über $p = 1..6$ höchstens **10 Einheiten**, während der Fit-Term sich schon bei 10 % Residuenunterschied um **19** und bei Faktor 2 um **139** ändert. AIC war hier faktisch eine monotone Transformation des Residuums — und Spearman ist gegen monotone Transformationen invariant. BIC wäre mit $\leq 26,5$ ebenfalls zu schwach gewesen.
4. **Abbruch** per vorab vereinbarter Regel — und wenig später **Wiederaufnahme**, weil die Falsifikation weich war: Die Intervention hatte die Rangfolge gar nicht bewegt, die Hypothese war also ungeprüft, nicht widerlegt.

3.11 WP-T1 — die Numerik-Diagnose (2026-07-22)

Bevor der zweite Screening-Versuch lief, wurde ein Confounder geprüft: Der finale Refit kehrte nach **5 Loss-Auswertungen** mit `retcode = Success` zurück, ohne etwas zu verbessern. Vermutung: Der Bewertungspfad simuliert mit `abstol = reltol = 1e-6` — Gradienten aus finiten Differenzen einer Größe, die nur auf 1e-6 genau berechnet wird, wären in diesem Bereich Rauschen.

Teil A — bestmöglich erreichbarer Loss mit den wahren Parametern:

Toleranz	System 3	System 11
1e-5	2,06e-10	4,79e-13
1e-6	4,40e-12	1,86e-14
1e-8	6,38e-16	1,36e-17
1e-10	4,58e-17	3,84e-19
1e-12	4,34e-17	3,86e-19

Baseline v0 meldet für System 11 einen Loss von 4,402e-15 — besser als das, was die wahren Parameter bei dieser Toleranz erreichen können (1,86e-14). Diese Zahl ist numerisches Rauschen.

Sie steht in Baseline v0, in der Phase-A-Auswertung und in allen bisherigen Regressionsprüfungen.

Teil B — Rauschgrenze des Optimierers. Erreichter Loss aus dem LS-Warmstart:

Toleranz	System 3	System 11
1e-6	3,236e-08	4,606e-15
1e-8	3,236e-08	1,669e-17
1e-10	3,236e-08	4,860e-18
1e-12	3,236e-08	4,856e-18

System 11 bestätigt die Vermutung (der Wert skaliert direkt mit der Toleranz). **System 3 widerlegt sie** — dort ist der Wert über sechs Größenordnungen völlig flach, obwohl der numerische Boden bei 4,40e-12 liegt, also 7.000-fach darunter.

Und damit war die eigentliche Ursache gefunden: 3,236e-08 ist exakt der Wert, bei dem die Screening-Variante hängenblieb. Der Referenzpfad erreicht 2,66e-10 und benutzt **keinen** Warmstart (`use_pretuning = false`). Der ableitungsbasierte Warmstart führt auf System 3 in ein Becken, aus dem BFGS nicht herausfindet — die Screening-Variante hatte also genau das wieder eingeführt, was die Konfiguration bewusst abgeschaltet hatte.

Zwei weitere Befunde nebenbei:

- **Pathologische Line-Search:** Einzelne Fits verbrauchen bei **zwei Parametern** bis zu **39.933 Loss-Auswertungen** (jede eine vollständige Integration) mit Retcode `Failure` , Laufzeiten 3,2 / 10,4 / 23,6 s — bei `maxiters = 200` , also rund 200 Auswertungen pro Iteration. Das erklärt die 7.281 Integrationen pro Fit aus der Regressionsmessung.
- **Sentinel-Loss:** Ein vollständig gescheiterter Fit meldet `final_loss = 1.000e+06` (der Initialwert `l_best`) mit Retcode `Success` und ist von einem echten schlechten Fit nicht unterscheidbar.

3.12 WP-P2.4 — der Durchbruch (2026-07-23)

Zwei getrennte Interventionen, jede einzeln messbar:

1. **Geschachtelter Modellvergleich als Gate** statt additivem Strafterm. Begründung: Kinder sind geschachtelte Obermengen ihrer Eltern, und für geschachtelte Least-Squares-Probleme ist das Residuum monoton nicht-steigend in der Termzahl — ein größeres Modell kann nie schlechter abschneiden. Ein Kind darf seinen Elternteil nur überholen, wenn die Residuenverbesserung größer ist, als ein zusätzlicher Parameter zufällig liefern würde.
2. **Polish-Start vom Screening entkoppelt:** Der LS-Fit dient nur noch der Bewertung, der Startpunkt fürs Nachpolieren kommt vom Referenzpfad.

System 3, Bewertungstoleranz 1e-8:

Bedingung	Zeit	Loss	Stage	Fits	Integrationen
A Referenz	78,6 s	6,25e-9	2	110	275.098
B Residuum + LS-Start (<i>vorher</i>)	353,2 s	3,236e-8	5	241	761.581
C Nested-Gate + LS-Start	49,2 s	3,236e-8	5	241	232.224
D Nested-Gate + entkoppelter Start	12,6 s	2,558e-9	2	61	73.680

D schlägt den Referenzpfad um Faktor 6,2 bei besserem Loss und korrekter Stage.

System 11: alle Bedingungen qualitativ gleichwertig (Loss ~2,0e-16, Stage 4, `pruned_match` true), D mit 0,6 s gegen 1,1 s am schnellsten.

Die Arbeitsteilung ist sauber getrennt:

- Das **Gate** senkt Kosten und repariert das Ranking: B → C bringt 7,2× bei identischem Ergebnis, 147 von 400 Kindern scheitern am Gate, die Auswahl weicht in 126 Fällen auf 7 von 20 Leveln ab, und die Rangübereinstimmung kippt von **-0,78 (Median -1,0)** auf **+0,26 (Median +0,48)**.
- Der **entkoppelte Startpunkt** repariert Qualität und Eskalation: C → D.

Keine der beiden Interventionen allein hätte gereicht.

Der Nebenfund mit möglicherweise größerer Tragweite: Der **Referenzpfad allein** profitiert schon von der engeren Toleranz.

System 3, Referenzpfad	Zeit	Loss	Stage	Overshoot
bei Toleranz 1e-6	279,3 s	2,66e-10	3	1
bei Toleranz 1e-8	78,6 s	6,25e-9	2	0

Erklärung: Bei 1e-6 erreicht der Optimierer auf Stage 2 die Schwelle `loss_tol = 1e-8` nicht zuverlässig, die Abbruchbedingung feuert nicht, und die Suche eskaliert.

Ein Teil des beobachteten Stage-Overshoots wäre damit ein numerisches Artefakt und keine Eigenschaft der Promotionsregel. Das berührt unmittelbar die 62-Prozent-Rechnung und die Begründung für v3. Bislang: eine Zelle, ein Seed, ein 1D-System — eine Hypothese, kein Befund.

4. Was implementiert ist und funktioniert

Komponente	Status
<code>discover()</code> Kernpipeline	stabil seit 2026-04-20
EvoGrow v1, v2.1, v2.2	stabil
EvoGrowV3 (gleichungsweise)	v3.3 Kindergenerierung + v3.4 Pro-Gleichungs-Promotion (<code>r_k</code>) + v3.5 Metriken; Lockstep-Anker erhalten
EvoGrowScreening	implementiert; nur Performance-Hebel, kein Discovery-Gewinn auf gekoppelten Systemen (WP-T2)
GPStructureSearch	Vergleichsbaseline
StagedPolynomialBasis , PolynomialBasis	stabil
BFGSOptimizer	+ deterministische Budgets, frühe Verwerfung, Retcode-Kategorien
<code>pretune.jl</code>	+ <code>derivative_screening_diagnostics</code> mit Gültigkeitsflag
Experimentinfrastruktur	Manifest, Runner, Aggregator, atomare Writes
Regressionshistorie	append-only, Fingerprint-basiert, resume-fähig
Kosteninstrumentierung	pro Level und pro Lauf, bis in die Records
Mikro-Benchmark	<code>studies/profiling/profile_eval_cost.jl</code>
Vergleichsskript Screening	<code>studies/debug/compare_screening_variant.jl</code> , vier Bedingungen
Numerik-Diagnose	<code>studies/numerics/solver_tolerance_noise_floor.jl</code>
Entscheidungsmessung System 26	<code>studies/numerics/system26_tolerance_screening.jl</code> , drei Bedingungen + Anker
v3 Pro-Gleichungs-Metriken	<code>eq_overshoot</code> , <code>eq_wasted_levels</code> ; Tests + gekoppelter Divergenz-Smoke

5. Was verworfen wurde — und warum

Verworfen	Grund	Beleg
Pretuning als Paper-1-Thema	Strategischer Fokuswechsel auf gestuftes Wachstum	2026-05-17
v2.2 als finale Paper-Variante	Systemweites Staging versagt auf gekoppelten Systemen	Gate 1, 3 von 5 Systemen <code>pruned_match = false</code>
<code>isoutofdomain</code> für frühe Verwerfung	Verwirft den Schritt und verkleinert <code>dt</code> , bricht nicht ab — teurer statt billiger	DiffEq-Quellen, <code>integrator_utils.jl:268-286</code>
Level-Budget-Kürzung auf 18	Loss bliebe identisch, aber Overshoot-Metriken würden verfälscht	Stage 5/Overshoot 2 bei 30 gegen Stage 3/Overshoot 0 bei 18 Leveln
AIC als Komplexitätsstrafe	Bei $n = 200$ vom Fit-Term dominiert; Strafe ≤ 10 gegen Fit-Differenz 19–139	WP-P2.3, Ergebnisse bit-identisch
BIC als Alternative	Ebenfalls zu schwach ($p \cdot \log n \leq 26,5$)	Rechnerisch, nicht getestet
LS-Warmstart als Polish-Startpunkt	Führt auf System 3 in ein Becken bei $3,236e-08$	WP-T1, flach über sechs Toleranz-Größenordnungen
Rangübereinstimmung nur über ausgewählte Kandidaten	Kann die eigene Frage nicht beantworten	WP-P2.2b-Review
Screening-Spur (zwischenzeitlich)	Abbruchregel ausgelöst — später revidiert, weil die Falsifikation weich war	2026-07-22, wieder aufgenommen 2026-07-23
Screening als Kern-Claim	Ranking-Kollaps (Spearman-Median $-0,014$) und Nested-Gate inert auf gekoppeltem System	WP-T2, System 26
Baseline-v1-Matrix mit 3 Varianten	v3-Spalte lief mit veraltetem v3.3-Code, screening irrelevant → 2/3 Wegwerf-Compute, ~2 Wochen	2026-07-30, abgebrochen bei 6/45

6. Die wichtigsten Zahlen an einem Ort

Kostenprofil (23 Zellen, 40,5 h): Stage 1 → 21 s/Level, Stage 5 → 878 s/Level (Faktor 42). 62 % der Rechenzeit oberhalb der nötigen Stage. 39 % nach dem Punkt, an dem der Loss bereits final war.

Bewertungspfad (System 26): 95 % Solve-Anteil, 4.707 Integrationen pro Parameter-Fit, 1,763 ms pro Integration, 153 s Overhead außerhalb der Integration.

Erreichte Beschleunigungen:

Maßnahme	Faktor	Gemessen auf
Solver-Budgets (WP-P1b)	2,71×	System 26, 370 Fits je Bedingung
Solver-Budgets, nur Stage 2	9,9×	System 26, je 8 Level
Nested-Gate (B → C)	7,2×	System 3
Bedingung D gegen Referenz	6,2×	System 3
Toleranz 1e-6 → 1e-8, Referenzpfad allein	3,6×	System 3

Numerische Grenzen (bestmöglicher Loss mit wahren Parametern): bei 1e-6 System 3 → 4,40e-12, System 11 → 1,86e-14. Der in Baseline v0 berichtete System-11-Loss von 4,402e-15 liegt darunter.

Entscheidungsmessung System 26 (WP-T2): Overshoot toleranzinvariant — R6 und R8 bit-identisch Stage 5 / Overshoot 2 / wasted 8. Der Overshoot ist algorithmisch. Suspend-fest: 8 von 25 Leveln, ~25 % der Integrationen jenseits Stage 3. Struktur: `du1` exakt getroffen, `du2` komplett falsch. Screening D8 gegen R8: 98.253 vs 3.348.287 Integrationen (34×), aber Ranking-Median -0,014, Gate inert.

Kosten der Baseline-v1-Matrix: ~16 h je gekoppelter Lauf; 45 Läufe ≈ ~2 Wochen; davon nur die 15 v2.2-Läufe verwertbar.

7. Offene Punkte, nach Dringlichkeit

- Scope-Entscheidung für die v2.2-Referenzbaseline — der aktuelle Engpass.** Alles Weitere (WP-v3.6) hängt daran. Zwei Optionen: (a) nur Varianten kürzen (45 → 15, ~Tage, misst genau das Bisherige), oder (b) zusätzlich `N_LEVELS` / `pretuning` für die Regression überdenken — schneller, ändert aber, was gemessen wird, und damit den `config_fingerprint`. Bewusst mit klarem Kopf zu entscheiden, nicht im Reaktionsmodus. Hygiene unabhängig davon: `BFGS_TIME_LIMIT_S` von 24 h auf Minuten senken (entschärft die Landmine, kein Speedup).
- WP-v3.6 — Validierung v3 gegen Baseline v1.** Braucht (1) und zusätzlich einen **frischen** v3-Lauf mit aktuellem Code (die v3-Spalte des abgebrochenen Laufs war veraltet). Kernfrage: senkt die gleichungsweise Promotion den Overshoot auf 26/31/63, jetzt pro Gleichung messbar (`eq_overshoot` / `eq_wasted_levels`)? Externer Langlauf — nur der User startet.
- Bewertungstoleranz.** WP-T2 hat die Frage verfeinert: Für den **Suchpfad auf gekoppelten Systemen** ist 1e-6 die günstigere und verhaltensgleiche Wahl (1e-8 senkt nur den Loss, ändert das Stopp-Verhalten nicht, verteuert aber die verschwendeten Spät-Stages). 1e-8 ist nur für exakt lösbare Systeme relevant, die die Toleranz tatsächlich erreichen (z. B. System 11) — separate, kleinere Frage.
- Belastbarkeit bisheriger Losses.** Der System-11-Wert (4,402e-15) ist Rauschen. Zu klären, welche weiteren berichteten Zahlen betroffen sind — auch in Phase A.
- Pathologische Line-Search** (bis zu 39.933 Auswertungen bei zwei Parametern) — unangetastet, aber ein erheblicher Kostenhebel.
- Sentinel-Loss 1e6 mit Retcode Success** — gescheiterte Fits sind nicht als solche erkennbar.
- Pruning-Schwelle** `1e-3 × max_coeff` möglicherweise zu streng (System 31 Seed 42, Spurious-Term `0,0022·u1`).

Seit dem letzten Stand erledigt:

- *Ist der Stage-Overshoot numerisch?* — **Beantwortet (WP-T2)**. Auf System 26 toleranzinvariant, also algorithmisch. Die v3-Begründung ist bestätigt, nicht bedroht. Auf System 3 war er numerisch; die Trennung „einfach = numerisch, gekoppelt = algorithmisch“ ist die stärkere Aussage.
 - *WP-v3.3–v3.5* — **geliefert**. Gleichungsweise Kindergenerierung, Pro-Gleichungs-Promotion, Metriken; die v3-Kette ist code-seitig komplett.
-

8. Der rote Faden

Das Projekt hat drei Ebenen, die sich in den letzten Tagen als verschränkt erwiesen haben:

Ebene 1 — die Forschungsfrage. Ist gestuftes, inkrementelles Wachstum ein guter Suchmechanismus? Gate 1 hat gezeigt: für gekoppelte Systeme in der v2.2-Form nicht. Daraus entstand v3 (gleichungsweises Staging). Diese Linie ist unverändert gültig und wartet bei WP-v3.3.

Ebene 2 — die Kosten. Die Suche war nicht nur langsam, sondern *falsch* langsam: 62 % der Rechenzeit gingen in Komplexität, die kein System brauchte. Overshoot und Kosten sind dasselbe Problem, weil jede zusätzliche Stage überproportional teurer ist. Das macht v3 zugleich zum Performance-Fix.

Ebene 3 — die Numerik. Zwischenzeitlich war offen, ob ein Teil der beobachteten Effekte gar nicht algorithmisch ist: Ein berichteter Loss lag unterhalb der Rechengenauigkeit, ein Optimierer meldete Konvergenz, wo er nur blind war, und die engere Toleranz allein hatte auf System 3 den Overshoot beseitigt. **Diese Frage ist mit WP-T2 beantwortet.** Auf System 26 — dem Gate-1-System — ist der Overshoot toleranzinvariant, also algorithmisch. Die Numerik erklärt den Overshoot auf *einfachen* Systemen, nicht auf den gekoppelten. Die Messgrundlage für v3 steht damit, und die saubere Trennung („einfach = numerisch, gekoppelt = algorithmisch“) ist ein stärkeres Paper-Argument als die ursprüngliche Vermutung.

Wo es jetzt hakt — Ebene 4, die Ökonomie der Evidenz. Die drei inhaltlichen Ebenen sind geklärt oder in Arbeit: v3 ist begründet (Ebene 1), v3 ist code-seitig fertig, Overshoot und Kosten sind dasselbe Problem (Ebene 2), die Numerik ist eingeordnet (Ebene 3). Was v3 noch fehlt, ist der **Nachweis am Vergleich** — und der scheitert derzeit nicht an der Wissenschaft, sondern an den Kosten: die Referenzbaseline in der aktuellen Form ist ~2 Wochen lang und zu zwei Dritteln Wegwerf-Rechnung. Das ist kein Rückschritt, sondern ein **Scope-Problem**: Wir müssen entscheiden, was die v2.2-Referenz kosten darf, bevor wir sie und den frischen v3-Lauf starten.

Was das für die nächsten Schritte heißt: Die nächste Entscheidung ist keine Messung, sondern eine Planungsfrage (7.1) — mit klarem Kopf, nicht im Reaktionsmodus. Ist sie getroffen, folgt genau ein sauberer Codex-Spec für den schlanken Runner, dann der externe v2.2-Lauf, dann der frische v3-Lauf, dann WP-v3.6. Die inhaltliche Unsicherheit ist ausgeräumt; offen ist nur noch, wie teuer wir den Beleg machen wollen.

3.13 WP-T2 — die Entscheidungsmessung (2026-07-29)

Die offene Frage aus 3.12 wurde beantwortet. Der Toleranzvergleich lief auf **System 26** — genau dem gekoppelten System, an dem Gate 1 gescheitert ist. Drei Bedingungen, Seed 42, 30 Level: R6 (Referenz, 1e-6), R8 (Referenz, 1e-8), D8 (Screening, 1e-8).

Vor dem Lauf wurde die Prognose **geschärft**: Auf System 3 war der numerische Kanal spezifisch — der Loss operiert nahe `loss_tol = 1e-8`, und bei $1e-6$ erreicht der Optimierer die Schwelle nicht, also feuert der Abbruch nicht und die Suche eskaliert. Auf System 26 liegt der Loss-Boden bei $\sim 1,4e-3$, drei Größenordnungen **über** selbst der $1e-6$ -Toleranz. `loss_tol` kann dort nie feuern, unabhängig von der Toleranz. **Vorhersage: die engere Toleranz ändert den Overshoot auf System 26 nicht — er ist hier algorithmisch.**

Anker bit-exakt reproduziert: R6 liefert `0.001391623174905009`, `final_stage = 5`, Overshoot 2, `wasted_levels = 8`, `pruned_match = false` — identisch zu Baseline v0. Damit ist die Messung interpretierbar.

Ergebnis — Vorhersage bestätigt: R6 und R8 haben bit-identisch `final_stage = 5`, `stage_overshoot = 2`, `wasted_levels = 8`. Die engere Toleranz senkt nur den Loss ($1,39e-3 \rightarrow 2,52e-4$), nicht das Stopp-Verhalten; beide terminieren via `plateau_absolute` und nähern sich `loss_tol` nie.

Der Overshoot auf System 26 ist algorithmisch, nicht numerisch. Damit trennt die Messung sauber: auf einfachen Systemen (System 3) numerisch, auf gekoppelten algorithmisch. Das ist eine *stärkere* Paper-Aussage als „Overshoot ist numerisch“ — und die v3-Begründung ist bestätigt, nicht bedroht.

Der v3-Beleg steckt in der gefundenen Struktur. Wahrheit: $du1 = 3 \cdot u1 - u1^2 - 2 \cdot u1 \cdot u2 \mid du2 = 2 \cdot u2 - u1 \cdot u2 - u2^2$. R8 nach Pruning trifft `du1` **exakt** ($3 \cdot 0,3 \cdot u1 - 1 \cdot 0,7 \cdot u1^2 - 1 \cdot 99 \cdot u1 \cdot u2$), aber `du2 = {u1, u1^2}` ist **komplett falsch**. Eine Gleichung gelöst, die andere im Blindflug — und der globale Plateau-Mechanismus eskaliert Stage 4/5 für **beide**, obwohl Gleichung 1 längst fertig ist. Das ist exakt die Signatur, die v3 (gleichungsweise Promotion) auflösen soll: gelöste Gleichung einfrieren, nur die offene weiterwachsen.

Screening auf gekoppeltem System: schnell, aber kein Discovery-Gewinn. D8 gegen R8 (beide $1e-8$): D8 braucht **98.253 Integrationen gegen 3.348.287** ($34\times$ weniger), aber beide `pruned_match = false`, beide Overshoot 2. Zwei Befunde entwerfen den Geschwindigkeitsvorteil als Discovery-Hebel:

- **Ranking-Kollaps:** `rank_agreement_spearman` Median **-0,014**. Das FD-Ableitungs-Screening rankt Kandidaten auf dem gekoppelten System praktisch nicht wie der echte Loss. Der $34\times$ -Vorteil kommt aus *wenig integrieren*, nicht aus gutem Diskriminieren.
- **Nested-Gate inert:** `selection_diff_from_residual = 0` über alle Level. Der WP-P2.4-Durchbruch, der auf System 3 das Ranking reparierte, ändert auf System 26 keine einzige Auswahl. Der Gate-Nutzen ist systemabhängig.

Fazit: Screening ist eine **Performance-Optimierung**, kein Discovery-Qualitäts-Hebel. Es gehört als optionale Beschleunigung dokumentiert, nicht in den Kern-Claim; der Ranking-Kollaps muss in die Discussion.

Messvorbehalt — die Wall-Clock ist kontaminiert. Der PC wurde während des Laufs $2\times$ zugeklappt (Arbeitsweg, je ~ 45 min) und es lief Nebenlast. Das verfälscht **ausschließlich** die Zeitachse (`elapsed_s`, `s/Level`, der zitierte $\sim 6\times$ -Speedup) — diese Zahlen dürfen nicht als präzise Messwerte gelten. Die tragenden Schlüsse stehen trotzdem, weil sie auf **Zählungen** ruhen, nicht auf Zeiten: der Anker ist bit-exakt (Determinismus ungebrochen), `time_limit_s` war nie bindend ($\max 31.413\text{ s} \ll 86.400\text{ s}$, keine Iteration abgeschnitten), und Stage/Overshoot/Struktur/Integrationszählungen sind

maschinenlastunabhängig. Konsequenz fürs Paper: **Kosten über Integrationszählungen berichten, nicht über Wall-Clock.**

Eine eigene Korrektur gehört hierher: Die früher genannten „62 % Overshoot-Kosten“ waren die *Wall-Clock*-Sicht. Suspend-fest und deterministisch sind es auf System 26 **8 von 25 Leveln jenseits der erwarteten Stage 3, ~25 % der Integrationen** (Stages 4+5: 832.350 von 3.348.287 Solves). Die Differenz zur Wall-Clock-Zahl kommt daher, dass späte Integrationen einzeln teurer sind — was teils echte Numerik ist, aber eben auf der kontaminierten Achse liegt.

3.14 EvoGrow v3 wird scharf geschaltet (2026-07-29 bis 2026-07-30)

Mit der bestätigten Begründung wurde die v3-Kette in regressionssicheren Schritten umgesetzt. Jeder Schritt wurde **statisch** geprüft (Julia nicht gestartet, um die zu dem Zeitpunkt noch laufende Baseline nicht um CPU zu bringen); die Ausführungs-Evidenz liefert jeweils der Test-/Smoke-Lauf.

- **WP-v3.3 — gleichungswise Kindergenerierung** (`src/structure/evogrow_v3_childgen.jl`). Neu freigeschaltete Terme werden pro Gleichung anhand ihres eigenen Stage-Zustands zugelassen; echte Kreuzterme über die Paarregel `min(eq_stages[v] for v in vars(t)) >= stage(t)` . Entscheidend: bei **uniformen** Stages — dem Lockstep-Zustand — delegiert der Pfad an den unveränderten v2.2-Code mit identischer RNG-Reihenfolge, ist also **strukturell bit-identisch**. Der Pro-Gleichungs-Pfad ist bis v3.4 im echten Lauf tot.
- **WP-v3.4 — gleichungswise Promotionsregel** (`src/structure/evogrow_v3_promote.jl`). Das Fortschrittssignal ist das **Ableitungsresiduum pro Gleichung** `r_k` , ausgewertet auf der beobachteten Trajektorie (`estimate_derivatives` aus `pretune.jl` , keine Integration im Normalpfad, RNG-neutral). Promotion pro Gleichung an drei Bedingungen (Budget, `r_k` -Plateau, `r_k > loss_tol`) plus Maxstufen-Guard; die globale Termination wird **vor** der Promotion geprüft, damit der `loss_tol` -Stopp Vorrang behält. Das ist der Schritt, der die v2.2-Gleichheit **bewusst bricht** — der Bit-Identitäts-Smoke wurde durch einen Scalar-Promote-Smoke ersetzt, und die Verifikation ist deterministische Unit-Test-Logik statt Bit-Gleichheit (alle vier Promotionsbedingungen einzeln, plus der `r_k` -Signaltest).
- **WP-v3.5 — Pro-Gleichungs-Metriken + Integrationsnachweis** (`eq_overshoot` , `eq_wasted_levels`). Rein abgeleitete Metriken, in den Regressions-Record aufgenommen (`nothing` bei Nicht-v3-Varianten, wie `eq_final_stages`), ohne `config_fingerprint` - oder Verhaltensänderung. Der zweite Zweck war das Schließen einer **Integrationslücke**: der divergente Pfad — v3.3-Kindergenerierung und v3.4-Promotion im Zusammenspiel — war bis dahin nur mit injizierten Stages unit-getestet, **nie end-to-end in einem echten gekoppelten Lauf**. Ein billiger synthetischer 2D-Smoke (eine Gleichung exakt linear → bleibt Stage 1, die andere braucht Stage 2 → steigt) lässt die Stages erstmals real divergieren und testet die neuen Metriken zugleich.

Damit ist die v3-Kette **code-seitig komplett** (v3.2 Brücke → v3.3 → v3.4 → v3.5). Was aussteht, ist die wissenschaftliche Validierung (WP-v3.6): zeigen, dass v3 den Overshoot auf den gekoppelten Systemen 26/31/63 gegenüber v2.2 senkt. Diese Validierung braucht Vergleichsdaten — und genau daran hakt es (3.15).

3.15 Der zweite Kostenzusammenbruch — die Baseline-v1-Matrix (2026-07-30)

Für die Validierung wurde eine neue Regressionsbaseline gestartet (nötig, weil sich die Konfiguration seit Baseline v0 geändert hat — `time_limit_s` ist jetzt explizit, der Fingerprint umfasst neue Felder). Nach **~24 Stunden** war der Lauf bei **6 von 45 Läufen (~13 %)** und hing im ersten gekoppelten System-26-Lauf, der

allein den Großteil des Tages fraß. Der Lauf wurde abgebrochen. Die Diagnose deckte ein Design- und ein Scope-Problem auf:

Der Runner ist zu groß. Die Matrix ist **3 Varianten** (`evogrow_v2_2_stage_local` , `evogrow_screening_derivative` , `evogrow_v3`) $\times$ 5 Systeme $\times$ 3 Seeds = **45 Läufe**. Bei `USE_PRETUNING = false` und `N_LEVELS = 30` kostet ein einzelner gekoppelter Lauf ~16 h (WP-T2-Anker). Über alle gekoppelten Zellen dreier Varianten summiert sich das auf **~1,5–2 Wochen** durchgehend.

Und ein großer Teil davon ist wertlos:

- Die `evogrow_v3` -Spalte lief mit **v3.3-Code** — der Julia-Prozess hatte den Code beim Start geladen, vor v3.4/v3.5. Diese Spalte ist damit Lockstep ($\equiv$ v2.2) und **kein** Beleg für das echte v3.4-Verhalten. Die 9 teuren gekoppelten v3-Läufe müssen für WP-v3.6 mit aktuellem Code ohnehin neu gefahren werden.
- Die `screening` -Spalte ist als Kern-Claim erledigt (3.13) und fürs Paper irrelevant.

Was tatsächlich gebraucht wird, ist genau **eine** Sache: die **v2.2-Referenz** unter dem neuen Fingerprint (15 Läufe, davon 9 teure gekoppelte).

Ein Nebebefund korrigiert eine frühere Überzeichnung: `BFGS_TIME_LIMIT_S = 86.400` (24 h pro Fit) ist zwar unangemessen für einen 45-Lauf-Runner, aber **nicht** der Kostentreiber — WP-T2 zeigt, dass kein Fit dem Limit nahekam. Die ~16 h je gekoppelter Lauf entstehen durch die *Anzahl* teurer Fits (`pretuning = false` , 30 Level, steife Kandidaten-ODEs), nicht durch einen hängenden Fit. Das Zeitlimit zu senken ist Hygiene, kein Speedup. Der einzige echte Hebel im Scope ist die **Varianten-Kürzung** (45 $\rightarrow$ 15).

Bewusst offen gelassen: ob die v2.2-Referenz „nur“ durch Varianten-Kürzung (~Tage, sicher) oder zusätzlich durch ein Überdenken von `N_LEVELS / pretuning` für die Regression billiger werden soll. Letzteres ändert, was gemessen wird, und wurde als Entscheidung mit klarem Kopf vertagt — nicht im Reaktionsmodus. Diese Entscheidung ist der aktuelle Engpass (siehe 7.1).